How different methods are able to understand language

Karson Arrants

Overview

The goal of this project was to analyze and test how different methods that we have learned throughout the semester are able to collect and “understand” the queries that we were given. In this, I went about three methods, one of which is the baseline for the project. A simple boolean search that is going to be used to compare to the other methods. Second, I used a form of rating search, using other columns from the datasheet to narrow down the amount. And finally, I used a parse tree as the ‘best’ method since it is the most complex when compared to the previous two. The point of all of these was not to have a perfect implementation, but rather to better understand how these methods can be used to properly teach a machine to understand language on a higher level. While there are also many others, some of which I tried to implement. I will only be focusing on these three

Background

To start this project, I have some experience with data science. So I wanted to see how some methods related to that might impact the data cleaning. Or checking how other columns might impact the relevancy. Though some results did end up surprising me.

Proposed methods

As mentioned above, the methods used here were Boolean Search, rating search, and a parse tree

Boolean Search (baseline)

This was the baseline method that I implemented, a simple AND/OR logic that would search through the database for any of the aspects, then an aspect AND the opinions. And finally, any of the aspects or opinions. This method had, in most cases, a much lower precision score. This was mostly due to many of the words in them being able to be used in sentences that had little or nothing to do with what I was looking for. Though what was interesting was to see how different queries had higher precision scores even in this. There is, of course, a degree of luck when picking out reviews to judge. My lowest score was for Mouse Button: Click Problem, which to me made sense since it was both the longest query and ‘problem’ is widely used in these reviews. Compared to my highest score, which was for Image Quality: Sharp. This was mostly because the opinion was mostly used when referring to pictures from a printer, camera, or TV screen. In the cases that it wasn't one of these. It was usually about a ‘quality’ ‘sharp’ knife which was pulled in. Overall, this method is important to establish a base to show just how wide these words can be used.

Rating Search

Next, I implemented a rating search, considering that data filtering and cleaning is an important part of data science. I wanted to see if monitoring the data, or in other words, forcing it to look at certain columns. In this case, first, I imported a list of positive and negative words. Which would take the opinion word and judge if it was positive or negative. Then it if it was positive, it would only look at the reviews that were greater than three stars. And if it were negative, the ones with fewer than three stars. Next, I also wanted to ensure that the person had actually bought the product. Then, finally what I looked at was taking the average of

the ‘helpful-review score’ and only considering reviews that landed above that amount. Since this was mostly me choosing the buckets, and how and what was allowed to be pulled out. It did somewhat improve the scores. Though this time my lowest score was Wifi Signal: Strong. Which actually experiences the smallest precision increase from the baseline, only a .001 difference, though the greatest difference was with my Mouse Button: Click Problem, which increased its score by .18. What I noticed here is that more reviews were close to the queries. But sometimes we're talking about an issue with a different product and not the one at hand. Meaning this method struggled to get much understanding at all from what it had. Which makes sense, it was not that much more complex than the baseline. Though it does show that even some minor changes to the data, removing more irrelevant reviews can certainly have an impact on the score.

Method 2: Parse Tree & Dependency

This method was the most complicated and resulted in improved scores across all of them. Though that didn’t come without some drawbacks that need to be considered. So firstly, how it works, the main thing I had to use here was spacy. Python's method of working with parse trees, mainly because it has dependency lists, and the other tools needed to properly make a dependency tree. When I first implemented it, my searching was far too strict and had a couple of logical errors. Though after some work, I was able to expand most of the reviews to have a larger recall. Most is the keyword, due to how my tree worked. And how many reviews were written. It had trouble with GPS MAP: USEFUL, only pulling out one result, which was correct. But relaxing it to increase the recall, in other words, search out farther in the tree and around to other sentences to get a deeper understanding. Here, I also used

something from my rating search. After I had my tree working, I implemented each of the different things for the rating search to see how they might impact it. Considering the ‘helpful review’ column didn't change the reviews it collected at all. Similarly with rating, this is likely due to the tree already being able to get an understanding from the sentences, and due to this, the added filter was not needed. Though that isn’t to say all of them had no use. Though I will get into this more later.

Design

The main thing that was used to design here, since most of the baseline and M1 were relatively simple. My M1 mostly just reuses a big portion from my baseline and essentially makes it more restrictive, and is mostly set that way from my input.Spacy, is a Python library that helps with tagging and tokenizing text. Similar to the Stanford Parser. The only one of my methods that really needs to be discussed, as far as how it works, is my M2. The main part of this function is where it searches through the tree, since by this point, spacy has already tagged and tokenized everything. First, after it finds an aspect word, it will search down in the tree, in other words, later in the sentence or review. .lemma basically is the way to look for similarly related words or phrase, then children looks for adjectives with nouns. Then, if nothing has been found there, it will then search up the tree, or in other words finding the action/verb combinations that might be relevant to the query. Sibling allows it to skip certain words to find the more relevant things, if there is a word in the middle that would be considered ‘noise’ or irrelevant to the review. Together, these allow it to find good results for most of the queries. The main issue was with GPS MAP: USEFUL. Which, I believe, is due to how the words are used within a sentence,

making it tough for the tree to detect it. I believe there might be some sort of solution to this, though I could not come up with a good way of doing it. And just to separate the M2, the 'dependency' part is all about the adjective, verbs, and all of those different ways to structure the sentence. While the Parse Tree part of this is merely how it looks around the given aspect to decide if it is worth being considered

Result

Now I can show all of the results from my testing, each data set that had a Retention of more than fifty, I would just read fifty of them and calculate the precision. Then scale the relevant amount

Query	Baseline (Boolean)		
	# Ret.	# Rel.	Prec.
idio quality:poor	1888	529	0.28
ifi signal:strong	310	120	.39
button:click problem	3552	49	.14
gps map:useful	346	132	.38
nage quality:sharp	1109	510	.45

found based on that precision score.

First, here are my results for my baseline, as you can see some of them came out to have quite large numbers, which makes sense since Mouse Button: Click problem has the most words. It also ends up having the lowest precision. And while the correlation isn't completely sound, there is a trend that relates a higher Retention to a lower precision score.

Next my M1, Rating Search:

Method 1 (M1)		
# Ret.	# Rel.	Prec.
159	48	.3
46	18	.391
182	16	.32
49	21	.42
232	139	.6

The amount of reviews collected from this method was vastly smaller than the baseline. Though it did have some interesting results compared to the baseline

Lastly is my M2, which had the smallest amount of retention so far. This also

resulted in a large precision score from them

Method 2 (M2)		
# Ret.	# Rel.	Prec.
52	28	.54
7	5	.71
19	12	.63
1	1	1
164	131	.8

all.

Discussion

For my baseline, it had the lowest precision. And while the correlation isn't completely sound, there is a trend that relates a higher retention to a lower precision score. Most of this issue is just because when looking for the relevant query, it searched for any aspect word and the opinion words. Which as shown, ends up creating a lot of noise and irrelevant results to what the query is.

Now my M1: Though surprisingly, in some cases, the precision barely increased, if at all. This could be due to the reviews I selected. But the more likely case is that Wifi Signal: Strong has a high chance of being a collocation, or sort of a bi-gram as these words seem to be together more commonly than some others. And just as in the baseline Image Quality: Sharp was the most precise of them all.

While my M2 did have the best results, that wasnt without some issues. I believe my precision was a little too high in some cases. This resulted in my parse tree being unable to detect certain queries very well from the reviews. The main culprit being GPS MAP: USEFUL. This was only able to collect one single result, upon looking deeper into why, I came to the conclusion that it has something to do with the grammar that is commonly used with these words. Wifi Signal: Strong, also faced this issue, but did better at finding them. I was not sure how to improve upon this method to increase the recall without completely ruining the precision of the

others, though something worth noting is that I did later end up adapting the part that verifies that the user purchased the items, which did have some small level of improvement over my method before I implemented this. One thing that did help somewhat with this issue was implementing a distance, in other words, how far I let the tree look around. The best I found was just using five, as going above that didn't really help much with each result.

Conclusion

From these methods, it can be seen how language is an intricate and deep mechanism for a machine to understand. In truth, no one method will be enough to completely understand a method. While my Parse Tree was the best at having a high Relevance, it also struggled to collect other reviews that would end up being considered relevant to this project. There is a balance that needs to be found between precision and recall, in order to get the most understanding. Likely looking at collocations and some that might have been related to the queries could have improved my results. But I believe this project helped me understand language better, and how it should be broken down into many parts; usually, the more it is broken down, the better results one can get from a dataset. If I were to continue this this I would likely try and implement more methods on top of one-another. And try to weigh them in a sense to ultimately get the most precise results while keeping a higher retention.

References

https://pypi.org/project/spacy/

https://www.geeksforgeeks.org/compiler-design/parse-tree-and-syntax-tree/